

# Materials Science Optimization Benchmark Dataset for High-dimensional, Multi-objective, Multi-fidelity Optimization of CrabNet Hyperparameters

Sterling G. Baird<sup>1,2\*</sup>, Xavier Zaitzeff<sup>2</sup>, Jeet N. Parikh<sup>3</sup>,  
Taylor D. Sparks<sup>1</sup>

<sup>1</sup>Materials Science & Engineering, University of Utah, 122 S. Central  
Campus Drive, #304, Salt Lake City, 84112, Utah, USA.

<sup>2</sup>Mechanical Engineering, Brigham Young University, 701 E University  
Parkway, 350 EB, Provo, 84602, Utah, USA.

<sup>3</sup>Northwood High School, 4515 Portola Pkwy, Irvine, 92620, California,  
USA.

\*Corresponding author(s). E-mail(s): [sterling.baird@byu.edu](mailto:sterling.baird@byu.edu);

## Abstract

Benchmarks are crucial for driving progress in scientific disciplines. To be effective, benchmarks should closely mimic real-world tasks while being computationally efficient, allowing for accessibility and repeatability. In material science and chemistry, relevant optimization tasks can be challenging due to their hierarchical, noisy, multi-fidelity, multi-objective, high-dimensional, and non-linearly correlated variables. Additionally, they may include mixed numerical and categorical variables subject to linear and non-linear constraints. This dataset is a hyperparameter optimization benchmark engineered to emulate the difficulty structure of such materials formulation problems. We generated 65,536 quasi-random combinations of 23 hyperparameters plus a training-set fraction and used them to train CrabNet, a composition-based property prediction model, on the Matbench experimental band gap task. Of 327,680 scheduled training runs, 173,219 completed (387 RTX-2080-Ti GPU days), giving 41,550 unique hyperparameter sets with 1 to 5 repeats each. The resulting regression dataset maps each hyperparameter combination to mean absolute error (MAE), root mean squared error (RMSE), computational runtime, and model size. The training-set fraction acts as a fidelity parameter: training on 30% of the

data costs about a third of the full runtime while preserving much of the ranking of hyperparameter sets. Repeated runs characterize heteroskedastic noise, and percentile ranks within each group of identical parameter sets are provided as a nonparametric description of that noise. Together with a released surrogate model, the dataset supports low-cost benchmarking of constrained, noisy, multi-objective, and multi-fidelity optimization algorithms.

**Keywords:** adaptive design, Bayesian optimization, formulation optimization, hyperparameter optimization, benchmark dataset, multi-fidelity

## 1 Background & Summary

Industry-relevant optimization tasks in the physical sciences are often “hierarchical, noisy, multi-fidelity [1–4], multi-objective [5–7], high-dimensional [8–11], and non-linearly correlated while exhibiting mixed numerical and categorical variables subject to linear [12] and non-linear constraints.” [12–14] Existing benchmark datasets [15–22], while very useful, typically are single-objective, single-fidelity, low-dimensional, and ignore or simplify the influence of noise. The aim of this dataset is to better model a physical science optimization task for benchmarking needs while properly quantifying the noise distribution. This approach bridges the gap between low-cost surrogate function evaluations using benchmark datasets and costly, real-world objective function evaluations by considering a multi-objective, multi-fidelity, and high-dimensional task while accounting for heteroskedastic noise.

The scope of the dataset should be stated plainly. It is a hyperparameter optimization benchmark built on a real materials machine learning task, engineered to emulate the difficulty structure of materials formulation problems (noise that changes across the search space, a cheap fidelity parameter, mixed variable types, several competing objectives, and optional composition-like constraints). It is not a materials design benchmark in which the inputs are chemical compositions or processing conditions. General hyperparameter optimization benchmarks such as HPOBench [23] and YAHPO Gym [24] cover many machine learning models; the dataset described here adds a materials property model, measured run-to-run noise from repeated training, and constraint modes that follow the geometry of composition constraints. Growing interest in self-driving laboratories [25, 26] and in newer composition-based models [27] increases the need for such realistic yet inexpensive test problems.

A typical materials researcher who wants to choose an optimization algorithm for an expensive campaign (for example, tuning a composition-based property model such as CrabNet, or planning a formulation study with Ax or BayBE) currently compares candidates on low-dimensional analytic test functions or on small tabular datasets, or simply uses a default. With this dataset, the same researcher can run many repeated campaigns against a surrogate of a real 24-input training process, with realistic noise, a fidelity parameter that trades accuracy for cost, and four objectives, at a cost of a fraction of a second per evaluation instead of minutes to hours of GPU time. The dataset and surrogate have already been used in this way: to compare Ax

and BayBE [28] strategies in public benchmarking notebooks<sup>1</sup>, through an interactive HuggingFace Space<sup>2</sup>, and in an Acceleration Consortium Kaggle competition<sup>3</sup>. We see it as the first entry in an extensible family of benchmarks: the same pipeline can be applied to other model architectures and Matbench tasks.

## 2 Methods

### 2.1 CrabNet, Matbench, and Ax

Predicting material properties is a difficult task, and elemental composition as a predictor of material properties while the specific structure of the material is unknown is a common problem [29–31]. CrabNet is a composition-based regression and classification algorithm for material properties using an attention-based neural network as a foundation [31]. CrabNet uses self-attention to featurize elemental composition data in the context of a chemical compound system. It has been shown to be effective in predicting material properties across several datasets and offers alternative visualizations of elemental behavior. The hyperparameters of CrabNet tune model aspects such as fitting parameters, training epochs, and network architecture, and tuning them is itself a high-dimensional optimization problem [8].

The data to train the CrabNet models was gathered from Matbench. Matbench is a collection of 13 machine learning tasks with 10 associated datasets to be used as a benchmark for novel algorithms [15]. Specifically for the benchmarking task of this paper, the data used for training the CrabNet models was taken from the experimental band gap task (`matbench_expt_gap`, 4,604 compositions with experimentally measured band gaps [32]), which uses composition only as input and is evaluated with five-fold nested cross-validation. We chose it because composition-only inputs match CrabNet’s design, the task is small enough to allow hundreds of thousands of training runs, and experimental targets make it a realistic materials task.

To ensure uniqueness and a uniform distribution across the search space, different hyperparameter combinations were generated by Ax platform’s quasi-random Sobol sampling function. Ax is a machine learning package for guided experimentation [33]. It offers intuitive APIs for sampling parameter spaces and Bayesian optimization for experiment generation across high dimensional mixed datatype parameters, built on BoTorch [34]. In this work, Ax was used only to generate the Sobol design; no Bayesian optimization was used to create the dataset. Sobol sequences are low-discrepancy (quasi-random) sequences rather than independent uniform draws: each marginal is uniform, while the points fill the joint space more evenly than independent draws [35].

### 2.2 Search space, fidelity parameter, and constraints

The search space contains 20 numerical hyperparameters, 3 categorical hyperparameters, and the training-set fraction `train_frac` (Table 2). A total of 65,536 ( $2^{16}$ ) Sobol points were generated, each was scheduled five times, and 173,219 CrabNet models were trained with the generated hyperparameter combinations

---

<sup>1</sup><https://github.com/AccelerationConsortium/baybe-multi-task-bo>

<sup>2</sup><https://huggingface.co/spaces/AccelerationConsortium/crabnet-hyperparameter>

<sup>3</sup><https://www.kaggle.com/datasets/acceleration-consortium/crabnet-optimization-challenge-dataset>

with the MAE, RMSE, model size, and run time recorded as outputs. Two linear constraints were enforced when generating the design and hold for every deposited row: `betas1`  $\leq$  `betas2`, and `emb_scaler` + `pos_scaler`  $\leq$  1. The second constraint is a genuine three-part mixture inside CrabNet, since the weight of the logarithmic fractional encoding is set to  $1 - \text{emb\_scaler} - \text{pos\_scaler}$ , so the three weights always sum to one.

`train_frac` is the only input that we frame as a fidelity parameter. Lower values train on a random subset of the training split, which lowers cost and accuracy. This mirrors common practice in large-scale machine learning, where hyperparameters are tuned on small, cheap proxy runs and then transferred to the expensive full-scale run [36], and multi-fidelity hyperparameter optimization methods that treat the training-subset size as a continuous fidelity [37]. Repeats of the same hyperparameter set are not a fidelity axis; they characterize noise (Section 2.4).

While there can be other uses, this dataset is primarily intended as a multi-objective, multi-fidelity, high-dimensional benchmark dataset for formulation-based optimization scenarios by scaling each of the numerical parameters to the range of 0 to 1 and applying a deliberately synthetic constraint that the sum of the parameters must equal one. For example, discovering new materials usually means elemental composition is known and other material properties are unknown until after manufacturing. As elemental composition needs to sum to one, constraining the numerical hyperparameters mimics the form of metal alloy composition constraints found in material science type problems. CrabNet hyperparameters analogously play the role of the composition that drives unknown structural changes, and by summing to one, they ensure that trade-offs can be seen between outputs. In context of the problem, it forces optimization algorithms to balance between independent variables for better outcomes.

Concretely, the constraint is an optional benchmark mode and is not applied to the deposited data. In the `PseudoCrab` benchmark classes of the `matsci-opt-benchmarks` package,  $n$  user-facing variables  $x_1, \dots, x_n \in [0, 1]$  ( $n = 5, 10$ , or  $20$  depending on the difficulty tier) are mapped to  $n$  of the numerical hyperparameters by min-max scaling, and a query is valid only if  $\sum_{i=1}^n x_i \leq 1$ . This inequality is equivalent to a sum-to-one equality with an implicit balance component, in the same way that the balance element of an alloy is often left implicit [12]. The constraint reproduces the simplex geometry of a composition space, which is what makes composition problems hard for many optimizers; it does not reproduce the physics, chemistry, or thermodynamics of real alloys. Our working hypothesis is that if a problem looks like a formulation problem (inputs on a simplex, trade-offs between components) and responds like one, it will also optimize like one. Two limits of this hypothesis follow from the design. First, uniform sampling of the region  $\sum x_i \leq 1$  places most variables near their lower bounds, a region that covers a fraction  $1/n!$  of the unit hypercube and therefore contains few or none of the deposited Sobol points when  $n$  is large, so benchmark results in this mode rely on the surrogate away from its training data. Second, the analogy concerns the shape of the search space, not the response surface, which comes from CrabNet training rather than from materials physics.

## 2.3 CrabNet training and execution environment

For each sampled combination, CrabNet v2.0.8 (<https://github.com/sparks-baird/CrabNet>) was trained and evaluated on each of the five folds of `matbench_expt_gap`, and the reported MAE and RMSE are averages over the folds. The number of training epochs is four times `epochs_step`, the widths of the four hidden layers of the output network are 8, 4, 2, and 1 times `out_hidden4`, and the number of attention heads is made even with `d_model` rounded to a multiple of it. To improve efficiency and reduce latency, hyperparameter sets (including repeats) were shuffled and divided into batches, then sent to the University of Utah’s Center for High Performance Computing (CHPC) through the SLURM scheduler using Submitit for asynchronous evaluation on RTX 2080 Ti GPUs. Despite some results not being completed due to either timeout or preemption, this trade-off was deemed reasonable for the efficiency and completion gains: 52.9% of the scheduled runs completed, and 63.4% of the Sobol points have at least one completed run. Results were logged in JSON format to a free-tier MongoDB Atlas database through the MongoDB Data API, then aggregated and prepared as machine-learning-ready datasets via Python in Jupyter notebooks. Only runs that finished and reported scores were exported; no other filtering was applied to the deposited table.

## 2.4 Aggregation and percentile ranking

To realistically capture noise in the benchmark dataset and avoid bias by assuming the noise distribution, simulations were repeated for each quasi-random parameter combination. The repeats were then ranked by the “dense” method with `pct=True` in `pandas.core.groupby.GroupBy.rank`, and the rank was added as a feature in the surrogate model for the nondeterministic outputs of MAE, RMSE, and runtime. Model size depends only on the architecture and does not vary between repeats. The rank is a nonparametric description of where a run falls among the repeats of its set; drawing a random rank at prediction time gives a noisy surrogate output. Section 4 compares this treatment with Gaussian noise models.

## 3 Data Records

The dataset and accompanying trained surrogates are deposited on Zenodo under a CC0 1.0 public domain dedication with the persistent identifier 10.5281/zenodo.7694268<sup>4</sup>. The deposition contains the files listed in Table 1. The primary data record is `sobol_regression.csv`; its columns are described in Table 2.

The record is designed to support the FAIR principles [38]. It is findable through a persistent DOI with descriptive metadata, accessible through open download without authentication, interoperable through plain CSV and JSON files with the schema in Table 2, and reusable through an open license, the provenance described in Section 2, and the versioned code in Section 6. The pickled surrogates require the scikit-learn version stated above.

---

<sup>4</sup><https://doi.org/10.5281/zenodo.7694268>

**Table 1** Files in the Zenodo deposition (DOI 10.5281/zenodo.7694268).

| File                                                                 | Description                                                                                                                                  |
|----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>sobol_regression.csv</code>                                    | Primary data table: one row per completed CrabNet training run (173,219 rows, 35 columns; Table 2).                                          |
| <code>model_metadata.json</code>                                     | Metadata of the surrogate models, keyed by output ( <code>mae</code> , <code>rmse</code> , <code>model_size</code> , <code>runtime</code> ). |
| <code>surrogate_models.pkl</code>                                    | Pickled scikit-learn random forest surrogates (scikit-learn 1.0.1) fit on the full table, one per output.                                    |
| <code>cross_validation_models.0.pkl</code><br>to <code>_4.pkl</code> | Surrogates fit on five cross-validation splits of the table, for held-out evaluation of the surrogate.                                       |

## 4 Technical Validation

Figure 1 summarizes the deposited table. Among the 41,550 unique hyperparameter sets, the number of completed repeats ranges from 1 to 5 with a mean of 4.17 (Figure 1a). The value of approximately 2.6 given in the submitted version was the number of completed runs per scheduled Sobol point (173,219 / 65,536), which includes the 36.6% of points with no completed run. MAE ranges from 0.169 to 1.214 eV and RMSE from 0.440 to 1.703 eV, both less than one order of magnitude. Runtime spans about four orders of magnitude (2.8 s to  $3.4 \times 10^4$  s, with 31 runs longer than one hour), and model size spans about two (from  $2.0 \times 10^6$  to  $1.2 \times 10^8$  trainable parameters).

The four outputs show genuine trade-offs (Figure 2). MAE and RMSE are strongly correlated (Spearman  $\rho = 0.97$  on repeat-averaged data), so they act largely as one accuracy objective. Accuracy trades off against cost: MAE and RMSE are negatively correlated with runtime ( $\rho = -0.57$  and  $-0.61$ ), giving Pareto fronts with 65 and 52 non-dominated sets. Model size is nearly independent of accuracy ( $|\rho| \leq 0.07$ ) and weakly correlated with runtime ( $\rho = 0.37$ ). The released surrogate, evaluated at the median noise rank, reproduces every pairwise correlation to within 0.02, which supports its use in place of the raw data.

The fidelity parameter behaves as intended (Figure 3). For 5,000 deposited hyperparameter sets, we used the surrogate to predict MAE and runtime at several values of `train_frac`. Rankings of the sets at reduced fidelity agree with those at full fidelity (Spearman  $\rho = 0.77$  at `train_frac` = 0.1, 0.84 at 0.3, and 0.93 at 0.5), while the median runtime falls to 14%, 33%, and 56% of the full-fidelity value. About half of the best 10% of sets at `train_frac` = 0.3 are also in the best 10% at full fidelity. Low-fidelity evaluations are therefore informative but imperfect, which is the setting multi-fidelity optimizers are designed for.

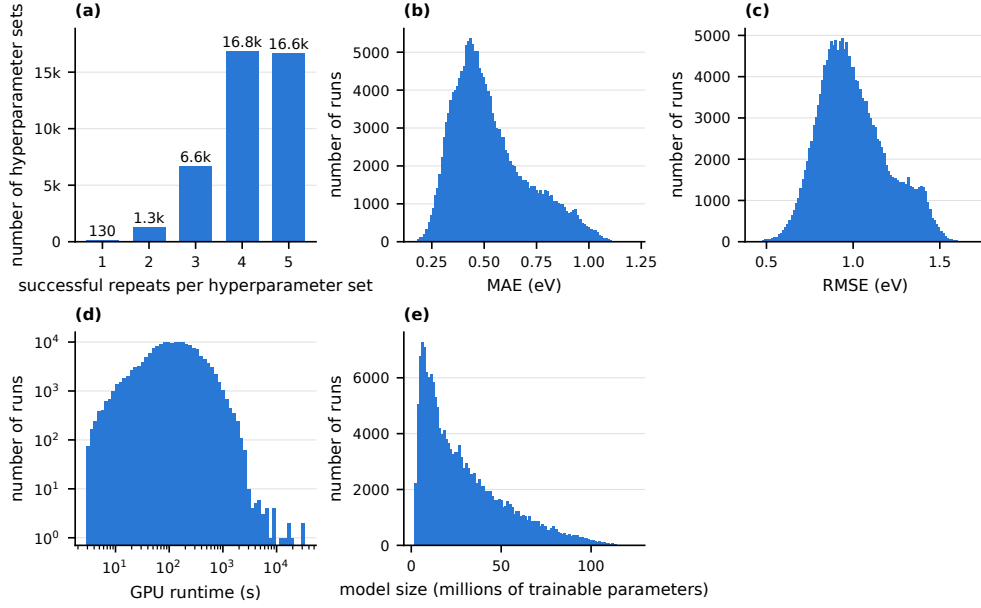
Repeated runs show that the noise is heteroskedastic. Within a hyperparameter set, the standard deviation of MAE varies 14-fold between the 5th and 95th percentiles of sets (0.003 to 0.047 eV) and that of runtime 190-fold (0.4 to 68 s), and both increase with the set mean (Spearman  $\rho = 0.33$  and 0.68). Standardized residuals are nearly symmetric for MAE (skewness 0.07) but skewed for runtime (0.68). To test the percentile-rank treatment, we split the sets with at least three repeats into 80% training and 20% held-out sets (8,025 held-out sets, 34,288 runs) and fit the same gradient-boosting learner to three noise models: a rank-conditioned model (the approach used for the released surrogate), a constant-variance Gaussian,

**Table 2** Data dictionary for `sobol_regression.csv`. Integer hyperparameters are marked (int). Bounds are those of the Sobol design.

| Column                                     | Type and range                                           | Description                                                                                                          |
|--------------------------------------------|----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <code>_id, session_id, timestamp</code>    | string, string, float                                    | MongoDB record identifier, batch session identifier, Unix time of the record (s)                                     |
| <code>N</code>                             | int, 1 to 10                                             | number of attention layers                                                                                           |
| <code>d_model</code>                       | int, 100 to 1024                                         | model (embedding) dimension                                                                                          |
| <code>dim_feedforward</code>               | int, 1024 to 4096                                        | width of the feed-forward sublayers                                                                                  |
| <code>heads</code>                         | int, 1 to 10                                             | number of attention heads                                                                                            |
| <code>dropout</code>                       | float, 0 to 1                                            | dropout probability                                                                                                  |
| <code>emb_scaler, pos_scaler</code>        | float, 0 to 1 each, sum $\leq 1$                         | weights of the element embedding and of the fractional encoding (the log fractional encoding receives the remainder) |
| <code>pe_resolution, ple_resolution</code> | int, 2500 to 10000                                       | resolution of the fractional and log fractional encodings                                                            |
| <code>fudge</code>                         | float, 0 to 0.1                                          | random perturbation of element fractions during training                                                             |
| <code>out_hidden4</code>                   | int, 32 to 512                                           | width of the last hidden layer of the output network                                                                 |
| <code>bias</code>                          | boolean                                                  | CrabNet bias flag                                                                                                    |
| <code>epochs_step</code>                   | int, 5 to 20                                             | training epochs divided by four                                                                                      |
| <code>batch_size</code>                    | int, 32 to 256                                           | batch size                                                                                                           |
| <code>lr</code>                            | float, $10^{-4}$ to $6 \times 10^{-3}$                   | learning rate                                                                                                        |
| <code>weight_decay</code>                  | float, 0 to 1                                            | weight decay                                                                                                         |
| <code>eps</code>                           | float, $10^{-7}$ to $10^{-4}$                            | optimizer epsilon                                                                                                    |
| <code>betas1, betas2</code>                | float, 0.5 to 0.9999, $\text{betas1} \leq \text{betas2}$ | optimizer beta coefficients                                                                                          |
| <code>alpha, k</code>                      | float 0 to 1, int 2 to 10                                | Lookahead optimizer step size and synchronization period                                                             |
| <code>criterion</code>                     | RobustL1, RobustL2                                       | loss function                                                                                                        |
| <code>elem_prop</code>                     | mat2vec, magpie, onehot                                  | element featurization                                                                                                |
| <code>train_frac</code>                    | float, 0.01 to 1                                         | fraction of each training split used (fidelity parameter); 16 rows from two small test sessions use 0.003            |
| <code>hardware</code>                      | string                                                   | GPU model (2080ti for all rows)                                                                                      |
| <code>mae, rmse</code>                     | float (eV)                                               | MAE and RMSE averaged over the five Matbench folds                                                                   |
| <code>runtime</code>                       | float (s)                                                | wall-clock time of the run (all five folds)                                                                          |
| <code>model_size</code>                    | int                                                      | number of trainable parameters                                                                                       |
| <code>mae_rank, rmse_rank</code>           | float, 0 to 1                                            | dense percentile rank of the output within the repeats of the same hyperparameter set                                |
| <code>runtime_rank</code>                  |                                                          |                                                                                                                      |

and a Gaussian whose standard deviation is predicted from the hyperparameters (Figure 4). By the continuous ranked probability score (CRPS) [39], the rank-conditioned model was best for runtime, whose noise is skewed (22.8 s, against 25.6 s for the heteroskedastic Gaussian and 34.0 s for the constant-variance Gaussian), while the heteroskedastic Gaussian was best for MAE (0.0166 eV, against 0.0171 eV and 0.0181 eV for the constant-variance and rank-conditioned models). The rank-conditioned intervals were too narrow for both outputs (their central 80% interval contained 37% of held-out MAE values and 23% of runtimes), because the ranks describe run-to-run spread but not the surrogate’s own error at unseen hyperparameters. The percentile ranks are therefore a useful nonparametric description of the measured repeats, most valuable for skewed outputs such as runtime, and users who need calibrated intervals should add an estimate of surrogate error.

To show how the benchmark separates optimization strategies, we ran repeated single-objective campaigns on the released surrogate (minimize MAE at `train_frac` =



**Fig. 1** Distributions in the deposited table. (a) Number of completed repeats per unique hyperparameter set (41,550 sets, mean 4.17). (b) MAE, (c) RMSE, (d) GPU runtime on one RTX 2080 Ti (both axes logarithmic, since runtime spans about four orders of magnitude), and (e) model size in millions of trainable parameters, over all 173,219 completed runs of CrabNet on the Matbench experimental band gap task.

1, both native constraints enforced, each observation drawn with a random noise rank, scored by the noise-free value at the median rank) with random search, grid search, Sobol and Latin hypercube [40] designs, a genetic algorithm [41], Ax with its default Bayesian optimization strategy, Ax with SAASBO [9], and BoFire [42] (Figure 5). The space-filling designs and the genetic algorithm ran five campaigns of 50 evaluations each; the Bayesian optimization campaigns are computationally heavier and reached 11 to 15 evaluations, so all strategies are also compared after 11 evaluations. After 50 evaluations, the best strategies reached a mean best MAE of 0.236 to 0.252 eV, grid search reached only 0.282 eV, and all remained well above the best deposited set (0.174 eV), so the problem is far from solved within a typical experimental budget. After 11 evaluations, Sobol sampling (0.268 eV) and Ax (0.274 eV) led and grid and random search trailed (0.306 eV), although the spread between campaigns is large at this budget.

## 5 Usage Notes

The primary table `sobol_regression.csv` can be loaded directly with `pandas.read_csv`. To use the dataset as a benchmark, we recommend the trained surrogate (`surrogate_models.pkl`), which maps any in-bounds hyperparameter vector and a noise rank to predicted MAE, RMSE, runtime, and model size. Drawing

the rank uniformly at random gives noisy evaluations; fixing it at 0.5 gives a noise-free reference value for scoring. Queries should respect the two native constraints, and `train_frac` should be treated as the fidelity parameter, with runtime as its cost. The optional sum constraint is available through the `PseudoCrab` classes of the `matsci-opt-benchmarks` package. The HuggingFace Space and the BayBE and Ax notebooks listed in Section 1 are working templates for new benchmarking studies; they are not part of the formal data record.

## 6 Code availability

All code used to generate, aggregate, and analyze the dataset is openly available in the `matsci-opt-benchmarks` repository (<https://github.com/sparks-baird/matsci-opt-benchmarks>), including the orchestration script `scripts/crabnet_hyperparameter/crabnet_hyperparameter_submitit.py`, the data preparation and surrogate fitting notebooks in `notebooks/crabnet_hyperparameter`, and the scripts that produce the figures in this article. The source state used to generate the dataset is archived on Zenodo as release v0.2.1 (<https://doi.org/10.5281/zenodo.7694289>). The CrabNet implementation used for training is at <https://github.com/sparks-baird/CrabNet> (version 2.0.8).

## CRedit author statement

**Sterling G. Baird:** Project administration, Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data Curation, Writing - Original Draft, Writing - Review & Editing, Visualization. **Xavier Zaitzeff:** Writing - Original Draft, Writing - Review & Editing. **Jeet N. Parikh:** Methodology, Software, Formal Analysis, Data Curation, Writing - Original Draft, Writing - Review & Editing. **Taylor D. Sparks:** Supervision, Funding acquisition.

## Acknowledgments

**Funding:** This work was supported by the National Science Foundation Division of Materials Research [Grant number DMR-1651668].

We thank Trupti Mohanty for discussion related to the future use-case of this dataset as an optimization benchmark. We acknowledge the University of Utah’s Center for High Performance Computing (CHPC) for providing computational resources. The code and manuscript associated with a benchmark dataset for particle packing simulations [13] served as a starting point for the code and manuscript for this work, for which there is shared language and document structure. We acknowledge OpenAI for providing free usage of their research tool, ChatGPT, which was used during the review and editing process, and Anthropic’s Claude, which was used to prepare revision analyses and edits.

## Declaration of interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## References

- [1] Ghoreishi, S.F., Molkeri, A., Arróyave, R., Allaire, D., Srivastava, A.: Efficient use of multiple information sources in material design. *Acta Materialia* **180**, 260–271 (2019) <https://doi.org/10.1016/j.actamat.2019.09.009>
- [2] Kandasamy, K., Vysyaraju, K.R., Neiswanger, W., Paria, B., Collins, C.R., Schneider, J., Póczos, B., Xing, E.P.: Tuning hyperparameters without grad students: Scalable and robust bayesian optimisation with dragonfly. *Journal of Machine Learning Research* **21**(81), 1–40 (2020)
- [3] Sabanza-Gil, V., Barbano, R., Gutiérrez, D.P., Luterbacher, J.S., Hernández-Lobato, J.M., Schwaller, P., Roch, L.: Best practices for multi-fidelity Bayesian optimization in materials and molecular research. *Nature Computational Science* **5**(7), 572–581 (2025) <https://doi.org/10.1038/s43588-025-00768-8>
- [4] Gantzler, N., Deshwal, A., Doppa, J.R., Simon, C.M.: Multi-fidelity Bayesian optimization of covalent organic frameworks for xenon/krypton separations. *Digital Discovery* **2**, 1937–1956 (2023) <https://doi.org/10.1039/D3DD00117B>
- [5] Hanaoka, K.: Comparison of conceptually different multi-objective Bayesian optimization methods for material design problems. *Materials Today Communications* **31**, 103440 (2022) <https://doi.org/10.1016/j.mtcomm.2022.103440>
- [6] Häse, F., Roch, L.M., Aspuru-Guzik, A.: Chimera: Enabling hierarchy based multi-objective optimization for self-driving laboratories. *Chemical Science* **9**(39), 7642–7655 (2018) <https://doi.org/10.1039/C8SC02239A>
- [7] Ament, S., Daulton, S., Eriksson, D., Balandat, M., Bakshy, E.: Unexpected improvements to expected improvement for Bayesian optimization. In: *Advances in Neural Information Processing Systems* 36 (NeurIPS) (2023)
- [8] Baird, S.G., Liu, M., Sparks, T.D.: High-dimensional Bayesian optimization of 23 hyperparameters over 100 iterations for an attention-based network to predict materials property: A case study on CrabNet using Ax platform and SAASBO. *Computational Materials Science* **211**, 111505 (2022) <https://doi.org/10.1016/j.commatsci.2022.111505>
- [9] Eriksson, D., Jankowiak, M.: High-dimensional Bayesian optimization with sparse axis-aligned subspaces. In: *Proceedings of the 37th Conference on Uncertainty in Artificial Intelligence (UAI)*. *Proceedings of Machine Learning Research*, vol. 161

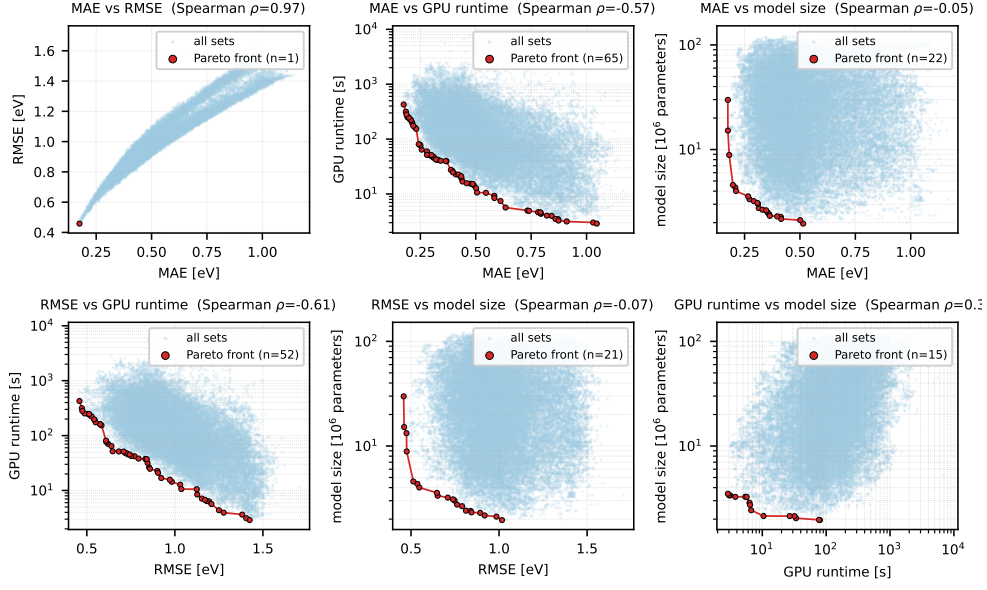
- (2021). <https://doi.org/10.48550/arXiv.2103.00349>
- [10] Hvarfner, C., Hellsten, E.O., Nardi, L.: Vanilla Bayesian optimization performs great in high dimensions. In: Proceedings of the 41st International Conference on Machine Learning (ICML). Proceedings of Machine Learning Research, vol. 235, pp. 20793–20817 (2024)
  - [11] Papenmeier, L., Nardi, L., Poloczek, M.: Bounce: Reliable high-dimensional Bayesian optimization for combinatorial and mixed spaces. In: Advances in Neural Information Processing Systems 36 (NeurIPS) (2023)
  - [12] Baird, S.G., Hall, J.R., Sparks, T.D.: Compactness matters: Improving Bayesian optimization efficiency of materials formulations through invariant search spaces. *Computational Materials Science* **224**, 112134 (2023) <https://doi.org/10.1016/j.commatsci.2023.112134> . Supersedes ChemRxiv preprints doi:10.26434/chemrxiv-2022-nz2w8-v2 and doi:10.26434/chemrxiv-2022-nz2w8-v3.
  - [13] Baird, S.G., Issa, R., Sparks, T.D.: Materials science optimization benchmark dataset for multi-objective, multi-fidelity optimization of hard-sphere packing simulations. *Data in Brief* **50**, 109487 (2023) <https://doi.org/10.1016/j.dib.2023.109487> . Supersedes ChemRxiv preprint doi:10.26434/chemrxiv-2023-fjjk7.
  - [14] Hickman, R.J., Tom, G., Zou, Y., Aldeghi, M., Aspuru-Guzik, A.: Anubis: Bayesian optimization with unknown feasibility constraints for scientific experimentation. *Digital Discovery* **4**, 2104–2122 (2025) <https://doi.org/10.1039/D5DD00018A>
  - [15] Dunn, A., Wang, Q., Ganose, A., Dopp, D., Jain, A.: Benchmarking materials property prediction methods: The Matbench test set and Automatminer reference algorithm. *npj Computational Materials* **6**(1), 138 (2020) <https://doi.org/10.1038/s41524-020-00406-3>
  - [16] De Breuck, P.-P., Evans, M.L., Rignanese, G.-M.: Robust model benchmarking and bias-imbalance in data-driven materials science: A case study on MODNet. *Journal of Physics: Condensed Matter* **33**(40), 404002 (2021) <https://doi.org/10.1088/1361-648X/ac1280>
  - [17] Wang, A.E., Liang, H., McDannald, A., Takeuchi, I., Kusne, A.G.: Benchmarking active learning strategies for materials optimization and discovery. *Oxford Open Materials Science* **2**(1), 006 (2022) <https://doi.org/10.1093/oxfmat/itac006>
  - [18] Liang, Q., Gongora, A.E., Ren, Z., Tiihonen, A., Liu, Z., Sun, S., Deneault, J.R., Bash, D., Mekki-Berrada, F., Khan, S.A., Hippalgaonkar, K., Maruyama, B., Brown, K.A., Fisher III, J., Buonassisi, T.: Benchmarking the performance of Bayesian optimization across multiple experimental materials science domains. *npj Computational Materials* **7**(1), 188 (2021) <https://doi.org/10.1038/s41524-021-00656-9>

- [19] Henderson, A.N., Kauwe, S.K., Sparks, T.D.: Benchmark datasets incorporating diverse tasks, sample sizes, material systems, and data heterogeneity for materials informatics. *Data in Brief* **37**, 107262 (2021) <https://doi.org/10.1016/j.dib.2021.107262>
- [20] Häse, F., Aldeghi, M., Hickman, R.J., Roch, L.M., Christensen, M., Liles, E., Hein, J.E., Aspuru-Guzik, A.: Olympus: A benchmarking framework for noisy optimization and experiment planning. *Machine Learning: Science and Technology* **2**(3), 035021 (2021) <https://doi.org/10.1088/2632-2153/abedc8>
- [21] Choudhary, K., Wines, D., Li, K., Garrity, K.F., Gupta, V., Romero, A.H., Krogel, J.T., Saritas, K., Fuhr, A., Ganesh, P., Kent, P.R.C., Yan, K., Lin, Y., Ji, S., Blaiszik, B., Reiser, P., Friederich, P., Agrawal, A., Tiwary, P., Beyerle, E., Minch, P., Rhone, T.D., Takeuchi, I., Wexler, R.B., Mannodi-Kanakkithodi, A., Ertekin, E., Mishra, A., Mathew, N., Wood, M., Rohskopf, A.D., Hattrick-Simpers, J., Wang, S.-H., Achenie, L.E.K., Xin, H., Williams, M., Biacchi, A.J., Tavazza, F.: JARVIS-leaderboard: A large scale benchmark of materials design methods. *npj Computational Materials* **10**, 93 (2024) <https://doi.org/10.1038/s41524-024-01259-w>
- [22] Hickman, R.J., Sim, M., Pablo-García, S., Tom, G., Woolhouse, I., Hao, H., Bao, Z., Bannigan, P., Allen, C.J., Aldeghi, M., Aspuru-Guzik, A.: Atlas: A brain for self-driving laboratories. *Digital Discovery* (2025) <https://doi.org/10.1039/D4DD00115J>
- [23] Eggenesperger, K., Müller, P., Mallik, N., Feurer, M., Sass, R., Klein, A., Awad, N., Lindauer, M., Hutter, F.: HPOBench: A collection of reproducible multi-fidelity benchmark problems for HPO. In: *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, vol. 1 (2021). <https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/93db85ed909c13838ff95ccfa94cebd9-Abstract-round2.html>
- [24] Pfisterer, F., Schneider, L., Moosbauer, J., Binder, M., Bischl, B.: YAHPO Gym: An efficient multi-objective multi-fidelity benchmark for hyperparameter optimization. In: *Proceedings of the First International Conference on Automated Machine Learning. Proceedings of Machine Learning Research*, vol. 188, pp. 3–139 (2022). <https://proceedings.mlr.press/v188/pfisterer22a.html>
- [25] Tom, G., Schmid, S.P., Baird, S.G., Cao, Y., Darvish, K., Hao, H., Lo, S., Pablo-García, S., Rajaonson, E.M., Skreta, M., Yoshikawa, N., Corapi, S., Akkoc, G.D., Strieth-Kalthoff, F., Seifrid, M., Aspuru-Guzik, A.: Self-driving laboratories for chemistry and materials science. *Chemical Reviews* **124**(16), 9633–9732 (2024) <https://doi.org/10.1021/acs.chemrev.4c00055>
- [26] Abolhasani, M., Kumacheva, E.: The rise of self-driving labs in chemical and materials sciences. *Nature Synthesis* **2**, 483–492 (2023) <https://doi.org/10.1038/>

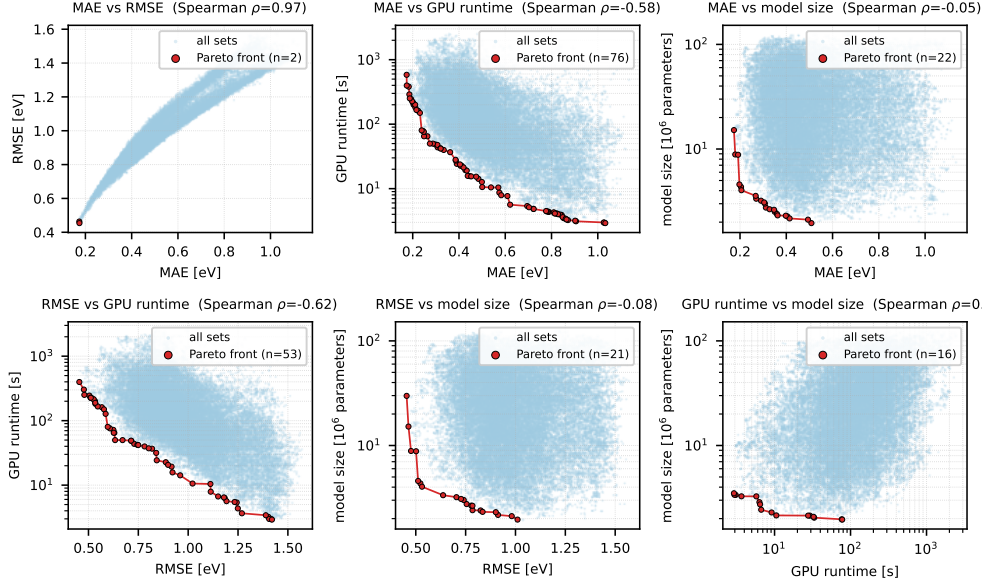
- [27] Madani, M., Lacivita, V., Shin, Y., Tarakanova, A.: Accelerating materials property prediction via a hybrid transformer graph framework that leverages four body interactions. *npj Computational Materials* **11** (2025) <https://doi.org/10.1038/s41524-024-01472-7>
- [28] Fitzner, M., Šošić, A., Hopp, A.V., Müller, M., Rihana, R., Hrovatin, K., Liebig, F., Winkel, M., Halter, W., Brandenburg, J.G.: BayBE: A Bayesian back end for experimental planning in the low-to-no-data regime. *Digital Discovery* (2025) <https://doi.org/10.1039/D5DD00050E>
- [29] Jha, D., Ward, L., Paul, A., Liao, W.-k., Choudhary, A., Wolverton, C., Agrawal, A.: ElemNet: Deep learning the chemistry of materials from only elemental composition. *Scientific Reports* **8**(1), 17593 (2018) <https://doi.org/10.1038/s41598-018-35934-y>
- [30] Goodall, R.E.A., Lee, A.A.: Predicting materials properties without crystal structure: Deep representation learning from stoichiometry. *Nature Communications* **11**(1), 6280 (2020) <https://doi.org/10.1038/s41467-020-19964-7>
- [31] Wang, A.Y.-T., Kauwe, S.K., Murdock, R.J., Sparks, T.D.: Compositionally restricted attention-based network for materials property predictions. *npj Computational Materials* **7**(1), 77 (2021) <https://doi.org/10.1038/s41524-021-00545-1>
- [32] Zhuo, Y., Mansouri Tehrani, A., Brgoch, J.: Predicting the band gaps of inorganic solids by machine learning. *The Journal of Physical Chemistry Letters* **9**(7), 1668–1673 (2018) <https://doi.org/10.1021/acs.jpclett.8b00124>
- [33] Olson, M., Santorella, E., Tiao, L.C., Cakmak, S., Garrard, M., Daulton, S., Lin, Z.J., Ament, S., Beckerman, B., Onofrey, E., Igusti, P., Lara, C., Letham, B., Cardoso, C., Shen, S.S., Lin, A.C., Grange, M., Kashtelyan, E., Eriksson, D., Balandat, M., Bakshy, E.: Ax: A platform for adaptive experimentation. In: *Proceedings of the Fourth International Conference on Automated Machine Learning. Proceedings of Machine Learning Research*, vol. 293, pp. 21–12125 (2025). <https://proceedings.mlr.press/v293/olson25a.html>
- [34] Balandat, M., Karrer, B., Jiang, D.R., Daulton, S., Letham, B., Wilson, A.G., Bakshy, E.: BoTorch: A framework for efficient Monte-Carlo Bayesian optimization. In: *Advances in Neural Information Processing Systems*, vol. 33, pp. 21524–21538. Curran Associates, Inc., Red Hook, NY, USA (2020). <https://proceedings.neurips.cc/paper/2020/hash/f5b1b89d98b7286673128a5fb112cb9a-Abstract.html>
- [35] Sobol', I.M.: On the distribution of points in a cube and the approximate

- evaluation of integrals. *USSR Computational Mathematics and Mathematical Physics* **7**(4), 86–112 (1967) [https://doi.org/10.1016/0041-5553\(67\)90144-9](https://doi.org/10.1016/0041-5553(67)90144-9)
- [36] Yang, G., Hu, E.J., Babuschkin, I., Sidor, S., Liu, X., Farhi, D., Ryder, N., Pachocki, J., Chen, W., Gao, J.: Tuning large neural networks via zero-shot hyperparameter transfer. In: *Advances in Neural Information Processing Systems*, vol. 34, pp. 17084–17097. Curran Associates, Inc., Red Hook, NY, USA (2021). <https://proceedings.neurips.cc/paper/2021/hash/8df7c2e3c3c3be098ef7b382bd2c37ba-Abstract.html>
  - [37] Klein, A., Falkner, S., Bartels, S., Hennig, P., Hutter, F.: Fast Bayesian optimization of machine learning hyperparameters on large datasets. In: *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics. Proceedings of Machine Learning Research*, vol. 54, pp. 528–536 (2017). <https://proceedings.mlr.press/v54/klein17a.html>
  - [38] Wilkinson, M.D., Dumontier, M., Aalbersberg, I.J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., Silva Santos, L.B., Bourne, P.E., Bouwman, J., Brookes, A.J., Clark, T., Crosas, M., Dillo, I., Dumon, O., Edmunds, S., Evelo, C.T., Finkers, R., Gonzalez-Beltran, A., Gray, A.J.G., Groth, P., Goble, C., Grethe, J.S., Heringa, J., 't Hoen, P.A.C., Hooft, R., Kuhn, T., Kok, R., Kok, J., Lusher, S.J., Martone, M.E., Mons, A., Packer, A.L., Persson, B., Rocca-Serra, P., Roos, M., Schaik, R., Sansone, S.-A., Schultes, E., Sengstag, T., Slater, T., Strawn, G., Swertz, M.A., Thompson, M., Lei, J., Mulligen, E., Velterop, J., Waagmeester, A., Wittenburg, P., Wolstencroft, K., Zhao, J., Mons, B.: The FAIR guiding principles for scientific data management and stewardship. *Scientific Data* **3**(1), 160018 (2016) <https://doi.org/10.1038/sdata.2016.18>
  - [39] Gneiting, T., Raftery, A.E.: Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association* **102**(477), 359–378 (2007) <https://doi.org/10.1198/016214506000001437>
  - [40] McKay, M.D., Beckman, R.J., Conover, W.J.: A comparison of three methods for selecting values of input variables in the analysis of output from a computer code. *Technometrics* **21**(2), 239–245 (1979) <https://doi.org/10.1080/00401706.1979.10489755>
  - [41] Blank, J., Deb, K.: pymoo: Multi-objective optimization in Python. *IEEE Access* **8**, 89497–89509 (2020) <https://doi.org/10.1109/ACCESS.2020.2990567>
  - [42] Dürholt, J.P., Asche, T.S., Kleinekorte, J., Mancino-Ball, G., Schiller, B., Sung, S., Keupp, J., Osburg, A., Boyne, T., Misener, R., Eldred, R., Costa, W.S., Kappatou, C., Lee, R.M., Linzner, D., Walz, D., Wulkow, N., Shafei, B.: BoFire: Bayesian Optimization Framework Intended for Real Experiments (2024). <https://doi.org/10.48550/arXiv.2408.05040> . <https://arxiv.org/abs/2408.05040>

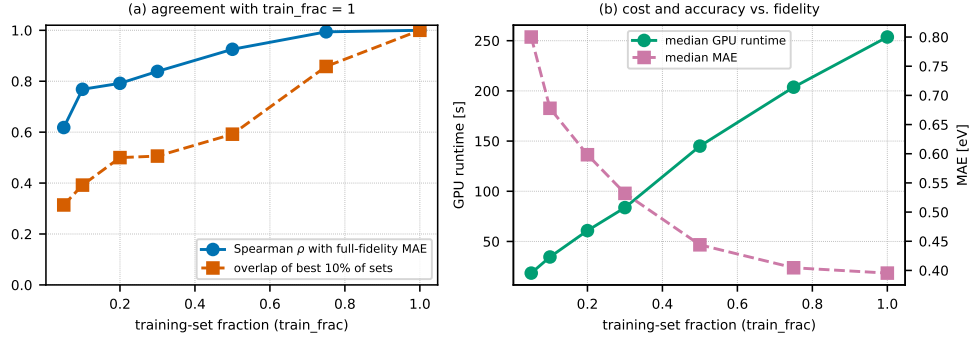
Raw data (objectives averaged over repeats)



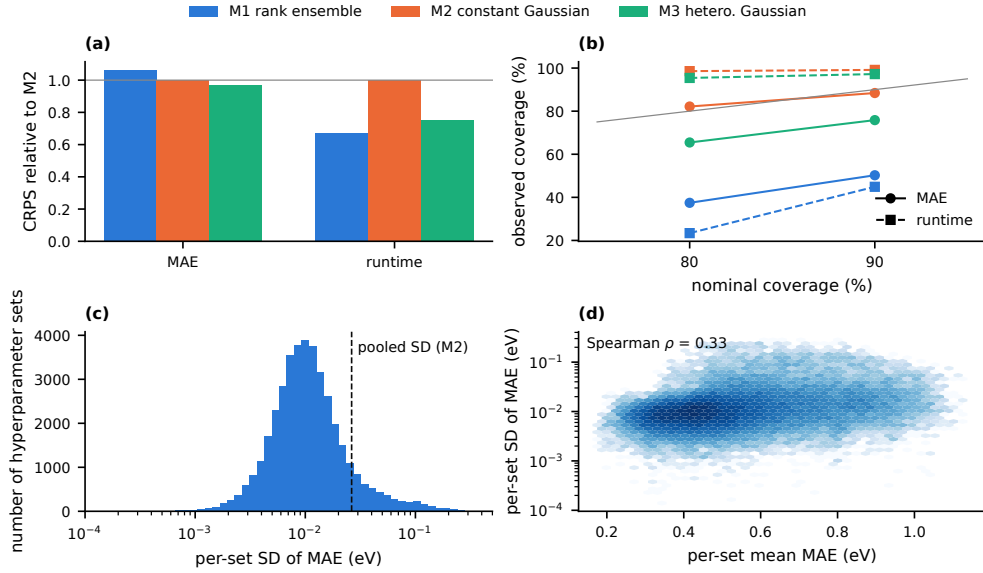
Surrogate model (median noise percentile, rank = 0.5)



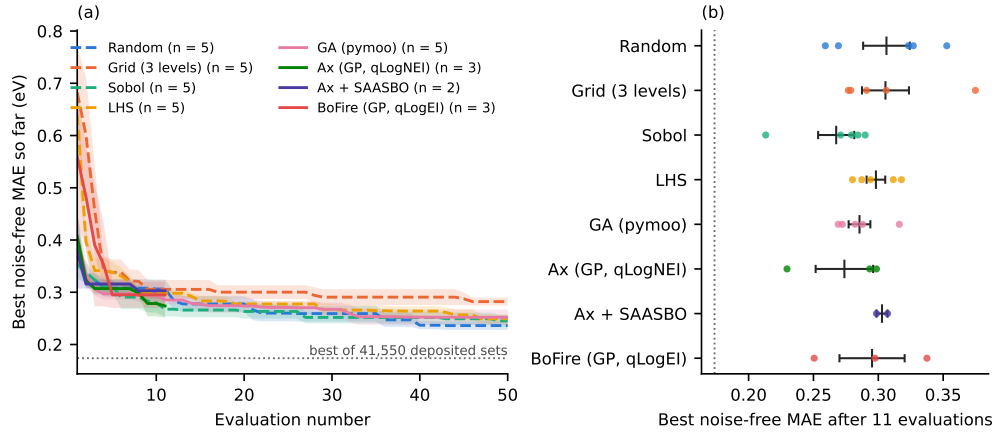
**Fig. 2** Pairwise Pareto fronts (red) of the four minimized outputs. Top six panels: raw data, averaged over the repeats of each of the 41,550 hyperparameter sets. Bottom six panels: the released surrogate evaluated at the same hyperparameter sets with the noise rank fixed at the median (0.5). Panel titles give the Spearman rank correlation.



**Fig. 3** The training-set fraction as a fidelity parameter, from surrogate predictions (median noise rank) for 5,000 deposited hyperparameter sets. (a) Spearman correlation between MAE at a given `train_frac` and MAE at `train_frac` = 1, and overlap of the best 10% of sets. (b) Median GPU runtime and median MAE as a function of `train_frac`.



**Fig. 4** Noise models compared on held-out hyperparameter sets (80/20 split by unique set; gradient-boosted tree models). M1: rank-conditioned model as in the released surrogate, with a predictive ensemble from 21 evenly spaced ranks. M2: Gaussian with a constant standard deviation equal to the pooled within-set value. M3: Gaussian with a per-set standard deviation predicted from the hyperparameters. (a) Mean CRPS relative to M2 (lower is better). (b) Observed coverage of central 80% and 90% intervals against nominal coverage. (c) Per-set standard deviation of MAE across repeats; the dashed line is the pooled value used by M2. (d) Per-set standard deviation of MAE against per-set mean MAE.



**Fig. 5** Optimization strategies on the released surrogate (minimize MAE, `train_frac` = 1, noisy observations). (a) Best noise-free MAE found so far (surrogate at median noise rank) against the number of evaluations, averaged over repeated campaigns ( $n$  in the legend; shaded band: standard error). Bayesian optimization campaigns end at 11 to 15 evaluations. (b) Best noise-free MAE after 11 evaluations for each campaign, with mean and standard error. The dotted line marks the best of the 41,550 deposited hyperparameter sets.